

Project Title: Share-k — AI-Powered Open Source Collaboration Hub

Team Name: Share-k Dev Team

Target Industry: Developer Tools & Productivity / Open Source / EdTech

Team Members: 6-Member Full Stack Team

1. The Core Problem (The "Why")

Open-source development powers the modern technology industry, yet it suffers from critical collaboration inefficiencies that harm both project maintainers and aspiring developers. Project owners receive floods of unqualified applications, while developers lack structured ways to discover matching projects, build verified reputations, or get actionable feedback when rejected.

Who suffers from this most:

- Project owners post contribution tasks but receive applications from developers who lack the required skills, wasting hours reviewing unsuitable candidates.
- Developers struggle to discover real projects that match their actual skill level, stifling career growth and portfolio development.
- There is no trusted, verified reputation system for contributor quality outside of raw GitHub metrics.
- Developers who are rejected receive no actionable guidance on how to improve or become eligible for future tasks.

2. The Solution & Value Proposition (The "What")

Share-k is an AI-powered open-source collaboration hub that eliminates these bottlenecks with a full-stack, AI-first platform. It uses LLM-driven skill profiling, RAG-powered contributor matching, and a multi-agent orchestration system to intelligently connect project owners with the right contributors — before a single application is sent.

The single most important benefit for the user: AI validates every contributor application BEFORE it reaches the project owner — eliminating wasted review time while creating a structured growth pathway for developers.

The platform solves the problem through four AI-driven pillars:

- Smart Skill Profiling: automatically analyzes GitHub repos and generates verified skill profiles (LLM + GitHub API + RAG).
- Intelligent Matching: matches contributors to tasks based on actual technical fit (Agentic AI + Vector Search).

- Validation Gating: blocks unqualified applications before they reach project owners (AI Skill Validation Agent).
- Growth Guidance: provides personalized learning paths to close skill gaps (RAG + LLM Generation).

3. The AI Trinity Strategy (The "How")

Share-k is built on three deeply integrated AI pillars that create a defensible technical moat and a superior developer experience:

LLM (Primary Examples): Foundation model: OpenAI GPT-4o (primary) + Anthropic Claude 3 Haiku (fallback). GPT-4o serves as the reasoning backbone of every AI agent — performing skill extraction from GitHub code, chain-of-thought eligibility validation, structured JSON skill profiling, and natural language guidance generation. Advanced API features used include function calling (tool use) in the Skill Validation Agent and token-by-token streaming for gap guidance reports. Claude 3 Haiku handles lower-stakes tasks such as notifications and summaries for cost efficiency.

RAG (Knowledge): The AI reads the following specific data — GitHub README files, code files (.py, .js, .ts, .go), commit messages, and repo descriptions for skill extraction. Contributor profiles indexed in Pinecone (vector DB) power contributor-to-task matching. A curated learning resource index of documentation summaries and tutorials fuels personalized skill gap guidance. Published project metadata enables semantic project discovery. Advanced RAG implementation uses hybrid retrieval (semantic vector search + BM25 keyword) with Reciprocal Rank Fusion, cross-encoder reranking (top-20 → top-5), and Query Transformation — decomposing task descriptions into targeted sub-queries for maximum retrieval recall.

Agents (Action): Four specialized agents run autonomously in a LangGraph sequential pipeline:

- GitHub Ingestion & Skill Profiling Agent — on user registration, autonomously fetches all GitHub repos, analyzes code with ReAct reasoning, and generates a structured JSON skill profile [{skill, level, confidence, source_repo}].
- Skill Validation Agent — when a contributor clicks 'Apply', autonomously compares their verified profile against task requirements using chain-of-thought reasoning, then issues an ELIGIBLE/INELIGIBLE decision with confidence score and justification — blocking unqualified applications before they reach the project owner.
- Skill Gap Guidance Agent (Gold Tier) — on rejection, autonomously retrieves curated learning resources and generates a personalized roadmap: {missing_skills, recommended_resources, practice_projects, estimated_weeks}.

- Contributor Matching Agent (Owner Gold/Silver) — when a new task is published, autonomously ranks the top-N best-fit contributors by skill alignment and notifies them proactively.

4. Market & Competition

Primary User: Two main user types:

- Project Owners — open-source developers and teams maintaining public repositories who need qualified contributors but lack a structured way to find and vet them.
- Contributors — junior to mid-level developers seeking real-world experience, portfolio projects, and a verified technical reputation beyond raw GitHub metrics.

The Advantage — Why Share-k is better than a simple search engine or existing non-AI software:

- Contributor Matching: Share-k uses AI-powered skill validation + semantic matching vs. manual review of all applicants on existing tools.
- Skill Verification: skills are AI-extracted from actual code, not self-reported as on other platforms.
- Rejection Experience: Gold tier contributors receive an AI learning roadmap generated instantly, while existing platforms provide no guidance.
- Reputation System: Share-k offers a multi-metric, contribution-verified public profile vs. star counts only on GitHub.
- Application Quality: only qualified applicants reach owners, whereas all applications reach owners unfiltered on existing tools.

5. Execution Plan (High Level)

Core Stack:

- Frontend: Next.js, Tailwind CSS, RTL
- Backend: Node.js — Python (FastAPI)
- Primary Database: MongoDB

Risk Factor — Biggest technical and data challenges expected:

- Skill extraction accuracy on sparse repos: mitigated by an Admin human review gate before profile activation; accuracy degrades gracefully and is flagged for manual assessment.

- GitHub API rate limits during bulk ingestion: mitigated by dynamic replanning in the Ingestion Agent, which queues retries for failed repos; OAuth increases rate limits significantly.
- LLM hallucination in skill profiles: mitigated by a faithfulness metric (>90% target) enforced via Langfuse; citation bubbles link each skill to its source repo as ground-truth evidence.
- LLM API cost at scale: mitigated by model tiering (GPT-4o only for high-value tasks), embedding caching, query caching, token budgets, and batch processing for matching jobs.
- Prompt injection in task descriptions: mitigated by Llama Guard screening on all user inputs, indirect injection defense in the GitHub README ingestion pipeline, and input sanitization middleware.